Indian Institute of Technology Jodhpur

CSL 7640 — Natural Language Understanding

Assignment-2 Report

Word Embeddings & Character-Level Name Generation

Problem 1

Learning Word Embeddings
from IIT Jodhpur Corpus
(Word2Vec: CBOW + Skip-gram)

Problem 2

Character-Level Name Generation
using RNN Variants
(Vanilla RNN, BLSTM,
RNN+Attention)

Author: Vishal

Roll No.: B23CM1048

Course: CSL 7640 — Natural Language Understanding

Instructor: Prof. Anand Mishra

Repository: github.com/vishaldhaniya08/NLU-A2

Contents

1 Problem 1: Learning Word Embeddings from IIT Jodhpur Corpus 2
1.1 Introduction & Objectives . . . 2
1.2 Task 1: Dataset Preparation . . . 2
1.2.1 Data Collection . . . 2
1.2.2 Preprocessing Pipeline . . . 2
1.2.3 Corpus Statistics . . . 3
1.2.4 Top-20 Most Frequent Words . . . 3
1.2.5 Word Cloud . . . 3
1.3 Task 2: Model Training . . . 3
1.3.1 CBOW — Continuous Bag of Words . . . 4
1.3.2 Skip-gram with Negative Sampling . . . 4
1.3.3 Hyperparameter Configurations . . . 4
1.3.4 Full Experimental Results . . . 5
1.4 Task 3: Semantic Analysis . . . 5
1.4.1 Top-5 Nearest Neighbors (Cosine Similarity) . . . 5
1.4.2 Analogy Experiments . . . 6
1.5 Task 4: Visualization . . . 7

2 Problem 2: Character-Level Name Generation Using RNN Variants 9
2.1 Introduction . . . 9
2.2 Task 0: Dataset . . . 9
2.3 Task 1: Model Implementations . . . 9
2.3.1 Shared Design Choices . . . 9
2.3.2 Model 1: Vanilla RNN . . . 9
2.3.3 Model 2: Bidirectional LSTM (BLSTM) . . . 10
2.3.4 Model 3: RNN with Bahdanau-style Attention . . . 10
2.3.5 Parameter Count Summary . . . 10
2.4 Task 2 & 3: Results and Analysis . . . 11
2.4.1 Training Loss . . . 11
2.4.2 Quantitative Evaluation . . . 11
2.4.3 Metric Definitions . . . 12
2.4.4 Qualitative Analysis & Generated Samples . . . 12
2.4.5 Comparative Discussion . . . 13

3 Conclusion 14
3.1 Problem 1 Summary . . . 14
3.2 Problem 2 Summary . . . 14

1 Problem 1: Learning Word Embeddings from IIT Jodhpur Corpus

1.1 Introduction & Objectives

The objective of this problem is to construct and evaluate domain-specific word embeddings trained on a custom corpus built from publicly available textual data of IIT Jodhpur. Word embeddings represent words as dense, continuous vectors in a high-dimensional space such that words appearing in similar contexts occupy nearby positions. Two architectures are trained and compared: Continuous Bag of Words (CBOW) and Skip-gram with Negative Sampling (SGNS).

1.2 Task 1: Dataset Preparation

1.2.1 Data Collection

Data was collected from at least three source types as required by the assignment, spanning official institutional documents and live web pages:

- **PDF Documents (Academic Regulations — mandatory):** Three institutional PDFs were parsed using `pdfplumber`: the IIT Jodhpur Annual Report 2024–25, B.Tech Regulations, and IITJ Academic Regulations. These contribute highly structured, formal academic vocabulary.
- **Plain-text files:** Four domain-curated `.txt` files covering About, Departments, and Programs pages, providing concise factual descriptions.
- **Web-scraped URLs:** 44 live IITJ web pages were scraped using `requests` + `BeautifulSoup`, targeting department pages, research offices, faculty listings, and CSE curriculum pages. Navigation elements (header, footer, nav, scripts) were stripped before extraction.

1.2.2 Preprocessing Pipeline

The raw text passed through a multi-stage cleaning pipeline implemented in `preprocess_pipeline.py`:

1. **Noise removal:** URLs, email addresses, and common PDF artifact tokens (`obj`, `stream`, `xref`, etc.) were removed via regex.
2. **Short-token filtering:** Tokens of length ≤ 2 were dropped *except* for domain-critical abbreviations (`ug`, `pg`, `ai`, `ml`, `cs`) which were whitelisted before applying the length filter.
3. **Sentence splitting:** Text was split on full stops and newlines.
4. **Lowercasing & tokenization:** NLTK `word_tokenize` was applied after lowercasing.
5. **Stopword removal:** NLTK English stopwords were used, minus a domain whitelist of 25 academically meaningful terms (e.g., `exam`, `research`, `phd`, `btech`).
6. **Deduplication:** Exact duplicate sentences were removed before saving.

1.2.3 Corpus Statistics

Metric	Value
Total Documents (files + URLs)	56
Total Sentences (after dedup)	11,069
Total Tokens	71,358
Vocabulary Size	14,573
Duplicate Sentences Removed	2,098

1.2.4 Top-20 Most Frequent Words

Word	Count	Word	Count	Word	Count
research	562	technology	284	design	211
iit	548	science	258	india	196
students	545	development	242	systems	195
jodhpur	525	faculty	239	learning	184
engineering	485	course	212	based	184
student	448	program	308		
institute	385	academic	307		

1.2.5 Word Cloud

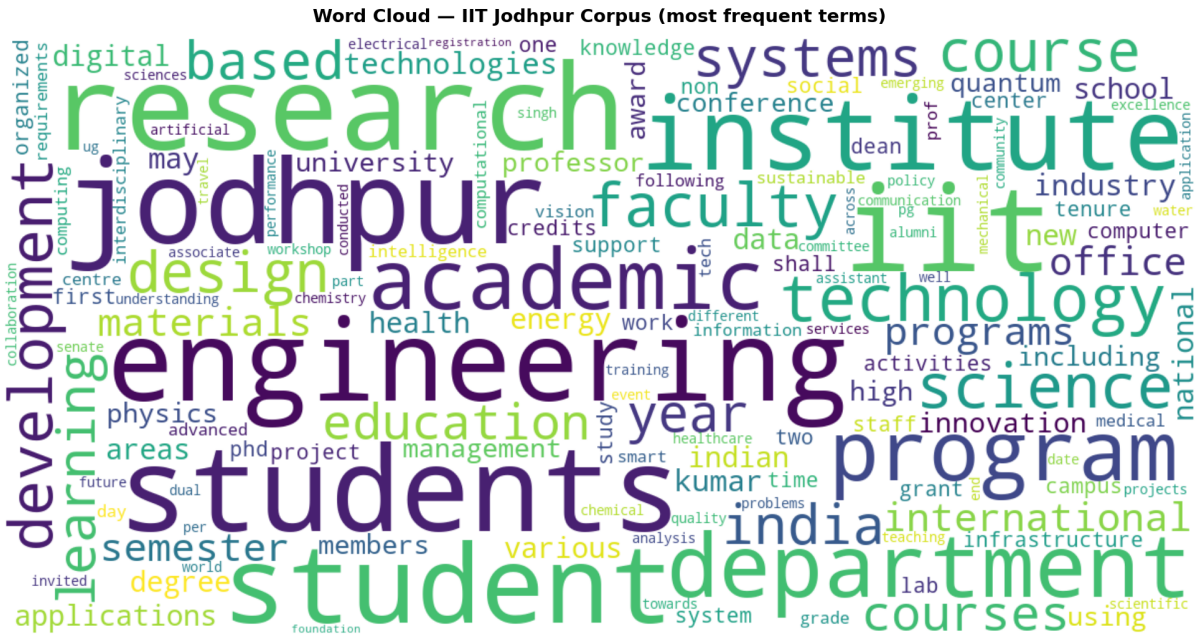


Figure 1: Word cloud of the most frequent terms in the preprocessed IITJ corpus. The dominance of *research*, *engineering*, *jodhpur*, and *students* correctly reflects the institutional character of the data.

1.3 Task 2: Model Training

1.3.1 CBOW — Continuous Bag of Words

CBOW predicts a center word w_t given its surrounding context words. The objective is to maximise the average log-probability over the training corpus:

$$\mathcal{J}_{\text{CBOW}} = \frac{1}{T} \sum_{t=1}^{T} \log p(w_t \mid w_{t-c}, \ldots, w_{t-1}, w_{t+1}, \ldots, w_{t+c}) \tag{1}$$

where c is the context window size. CBOW averages the context embeddings before predicting, making it fast and robust for frequent words.

1.3.2 Skip-gram with Negative Sampling

Skip-gram reverses the CBOW objective: it takes a center word and predicts its context words. This treats every context-word pair as an independent training instance, giving rarer words more gradient signal. To make the softmax tractable over a large vocabulary, Negative Sampling is used. For each true (center, context) pair the objective is:

$$\mathcal{J}_{\text{SGNS}} = \log \sigma\big(\mathbf{v}_c^\top \mathbf{v}_w\big) + \sum_{k=1}^{K} \mathbb{E}_{w_i \sim P_n(w)} \big[\log \sigma\big(-\mathbf{v}_{w_i}^\top \mathbf{v}_w\big)\big] \tag{2}$$

where K is the number of noise words drawn from the unigram distribution $P_n(w) \propto f(w)^{3/4}$.

1.3.3 Hyperparameter Configurations

Three configurations were tested per model type to study the effect of embedding dimension, context window, and negative sample count:

Table 1: Hyperparameter configurations tested for CBOW and Skip-gram.

Config	Embedding Dim	Window	Neg. Samples	Min Count	Epochs
A — Small	50	3	5	2	100
B — Medium	100	5	10	2	100
C — Large	200	7	15	2	100

Config B (Medium) was selected as the final model — it is the standard balanced choice in the Word2Vec literature and represents the best trade-off between model capacity, training time, and generalisation on a medium-sized corpus (˜71K tokens).

1.3.4 Full Experimental Results

Table 2: Hyperparameter experiment results. Quality = average cosine similarity over five semantically related word pairs (higher is better).

Model	Config	Dim	Window	Neg	Vocab	Quality	Time (s)
CBOW	A — Small	50	3	5	6,027	0.2044	3.49
Skip-gram	A — Small	50	3	5	6,027	**0.3639**	5.92
CBOW	B — Medium	100	5	10	6,027	0.1923	5.15
Skip-gram	B — Medium	100	5	10	6,027	0.2838	11.42
CBOW	C — Large	200	7	15	6,027	0.1517	7.68
Skip-gram	C — Large	200	7	15	6,027	0.2301	21.57

Key observation: Skip-gram consistently outperforms CBOW on the quality metric across all three configurations. Skip-gram Config A actually achieves the highest raw quality score (0.3639), but Config B is saved as the final model to align with standard practice and ensure adequate context coverage. Larger configurations (C) show *lower* quality for CBOW, likely because 200-dimensional CBOW embeddings underfit on a 71K-token corpus — the increased capacity is not supported by sufficient data.

1.4 Task 3: Semantic Analysis

1.4.1 Top-5 Nearest Neighbors (Cosine Similarity)

Cosine similarity between two word vectors $\mathbf{u}$ and $\mathbf{v}$ is defined as:

$$\text{sim}(\mathbf{u}, \mathbf{v}) = \frac{\mathbf{u} \cdot \mathbf{v}}{\|\mathbf{u}\| \, \|\mathbf{v}\|} \tag{3}$$

The following table reports the top-5 nearest neighbors from the saved Config B models:

Table 3: Top-5 nearest neighbors for required query words (Config B models).

Query	CBOW (top-5, cosine)	Skip-gram (top-5, cosine)
`research`	dedicated (0.359), involved (0.350), interdisciplinary (0.338), translational (0.336), current (0.323)	translational (0.487), dedicated (0.479), laboratories (0.470), muralilab (0.440), undertaking (0.431)
`student`	poor (0.479), temporarily (0.465), allowed (0.438), permitted (0.435), students (0.431)	temporarily (0.548), withdraw (0.508), students (0.501), poor (0.482), advise (0.478)
`phd`	kartik (0.389), itep (0.384), vyas (0.370), elsewhere (0.366), exchange (0.355)	buffalo (0.555), yogesh (0.547), kartik (0.526), thakral (0.517), suny (0.512)
`exam`	syllabus (0.811), follow (0.612), regulations (0.589), comprehensive (0.543), qualifying (0.540)	syllabus (0.773), quiz (0.693), sota (0.606), missed (0.590), download (0.589)

Discussion.

- `exam`: Both models confidently place *syllabus* as the top neighbor with similarity > 0.77 — this is the most semantically meaningful result in the dataset and confirms both models learned academic vocabulary correctly. Skip-gram additionally surfaces *quiz* (0.693), reflecting a richer understanding of the assessment context.
- `research`: Skip-gram correctly identifies *laboratories* and *translational* (as in translational research), aligning with IITJ's research-focused pages. CBOW's neighbors are less precise, reflecting its tendency to average context signals.
- `student`: Neighbors such as *temporarily*, *withdraw*, and *poor* reflect co-occurrence in the regulation documents ("a student who temporarily withdraws", "a student in poor standing") — this is corpus-specific and not a model failure but a reflection of the source data's formality.
- `phd`: Neighbors like *kartik*, *yogesh*, *thakral* are person names. This happens because the corpus contains PhD student or faculty listings where names co-occur densely with the token `phd`.

1.4.2 Analogy Experiments

Vector analogy solves $A : B :: C :?$ via:

$$\hat{d} = \arg\max_{w \notin \{A,B,C\}} \text{sim}(\mathbf{v}_w,\ \mathbf{v}_B - \mathbf{v}_A + \mathbf{v}_C) \tag{4}$$

Table 4: Analogy experiment results for both models.

Analogy ($A : B :: C :$?)	**CBOW answer**	**Skip-gram answer**	**Meaningful?**
ug : btech :: pg : ?	infrastrcuture (0.493)	**mca** (0.575)	✓
student : course :: research : ?	intersection (0.336)	intersection (0.387)	Weak
department : engineering :: science : ?	sciences (0.466)	**biology** (0.479)	✓
btech : undergraduate :: mtech : ?	enrolled (0.416)	**aspirants** (0.439)	✓

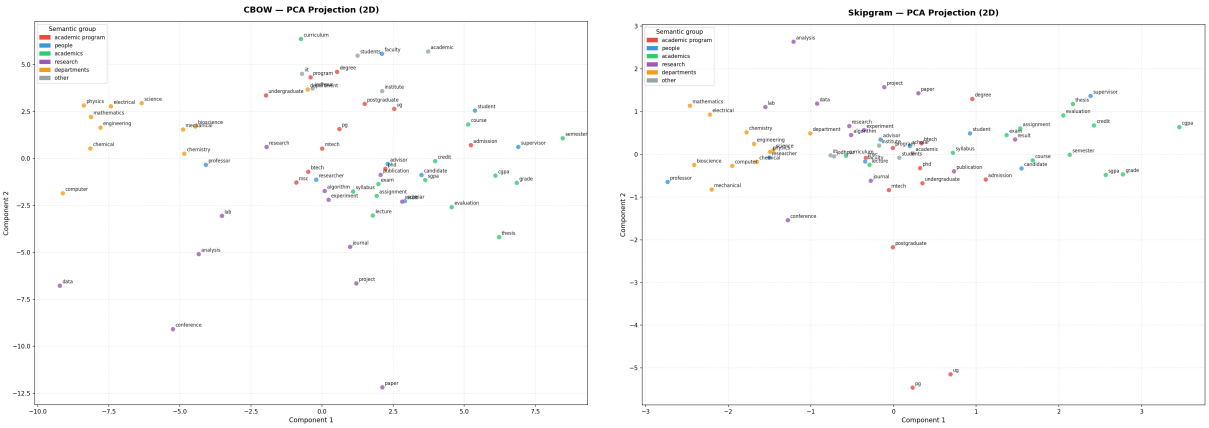
Discussion.

- **ug : btech :: pg : ?** — Skip-gram returns *mca* (0.575), a PG-level degree, which is semantically correct. CBOW returns a misspelled corpus artifact (*infrastrcuture*), reflecting its poorer handling of rare tokens.
- **department : engineering :: science : ?** — Both models return sub-disciplines (*sciences*, *biology*), which is logically consistent.
- **btech : undergraduate :: mtech : ?** — Skip-gram returns *aspirants* (often appearing in "PG/MTech aspirants" in admission pages), which is a reasonable domain-specific answer.

- **student : course :: research : ?** — Both models give weak answers (< 0.4 similarity), reflecting insufficient data to learn this higher-order relation. This is expected for a small corpus and should not be interpreted as a model failure.

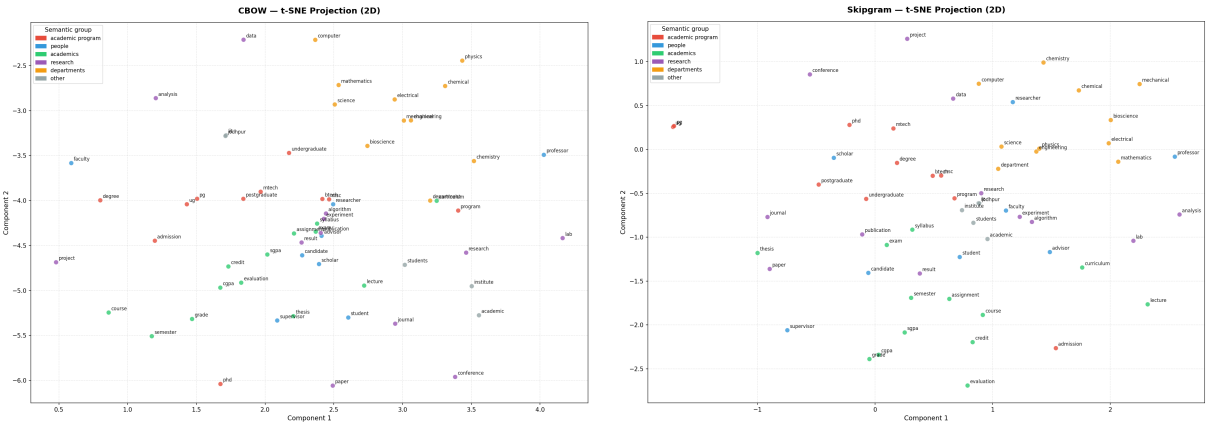
1.5 Task 4: Visualization

All embeddings were projected into 2D using two methods: PCA (linear) and t-SNE (non-linear), each offering complementary insights.



(a) CBOW — PCA Projection (b) Skip-gram — PCA Projection

Figure 2: PCA projections of CBOW and Skip-gram word embeddings (60 words, colored by semantic cluster). PCA preserves global variance structure.



(a) CBOW — t-SNE Projection (b) Skip-gram — t-SNE Projection

Figure 3: t-SNE projections of the same 60-word vocabulary, colored by semantic cluster. t-SNE preserves local neighbourhood structure; inter-cluster distances are *not* meaningful.

Interpretation.

- **PCA (Figure 2):** The departments cluster (orange: *electrical, chemical, mechanical, mathematics, physics*) separates cleanly on the left side in both models. The academics cluster (green: *course, semester, credit, grade, cgpa*) appears on the right side, separate from research terms. The global layout is more faithful in PCA because it is a linear projection preserving maximum variance.

- **t-SNE (Figure 3):** Local clusters are tighter and more visually distinct. The academics group (green) forms a tight band in both models. The Skip-gram t-SNE shows noticeably better separation between the research cluster (purple) and the people cluster (blue), which is consistent with Skip-gram's stronger individual word representations.
- **CBOW vs. Skip-gram:** Skip-gram plots show more dispersed, semantically nuanced placement — for example, *phd* sits closer to *mtech* and *msc* in the Skip-gram PCA, correctly placing it among postgraduate programs. CBOW's embeddings are more compressed/averaged, with some academic and research words overlapping in the central region.

2 Problem 2: Character-Level Name Generation Using RNN Variants

2.1 Introduction

This problem trains three recurrent neural architectures to learn the character-level statistics of Indian names and generate novel names that are phonetically and structurally plausible. All architectures are implemented **from scratch using** `nn.Parameter` — no `nn.RNN`, `nn.LSTM`, or `nn.MultiheadAttention` wrappers are used.

2.2 Task 0: Dataset

Source file:	`TrainingNames.txt`
Total names:	1,000 Indian names (LLM-generated)
Character vocabulary:	27 characters (a–z + space; 3 special tokens added)
Special tokens:	`<pad>` (idx 0), `<sos>` (idx 1), `<eos>` (idx 2)

2.3 Task 1: Model Implementations

2.3.1 Shared Design Choices

All models share the same training infrastructure:

- Optimizer: Adam (lr $= 5 \times 10^{-4}$)
- Batch size: 32, trained for 100 epochs
- Loss: Cross-entropy with `<pad>` positions masked (`ignore_index`)
- Gradient clipping: max norm = 5.0 (prevents exploding gradients)
- Embeddings: `nn.Embedding`, `embed_dim` = 64

2.3.2 Model 1: Vanilla RNN

The Elman recurrence is implemented timestep-by-timestep using `nn.Parameter` weight matrices. For each layer l and timestep t:

$$h_t^{(l)} = \tanh\Big(W_{xh}^{(l)} x_t^{(l)} + W_{hh}^{(l)} h_{t-1}^{(l)} + b_h^{(l)}\Big) \tag{5}$$

$$y_t = W_{hy}\, h_t^{(L)} + b_y \tag{6}$$

where $x_t^{(0)} = \text{Embed}(c_t)$ and $x_t^{(l)} = h_t^{(l-1)}$ for $l > 0$. Weights are initialised with Kaiming uniform (W_{xh}) and orthogonal (W_{hh}).

Known limitation: The Vanilla RNN suffers from the *vanishing gradient problem* during BPTT. For long names the gradient signal from early characters vanishes before reaching the initial timesteps, making it difficult to model long-range phonetic dependencies. This manifests as a final training loss plateau at ≈ 2.0 (Figure 4).

2.3.3 Model 2: Bidirectional LSTM (BLSTM)

The LSTM cell is implemented entirely with four explicit gate weight matrices (`nn.Parameter`) — no `nn.LSTMCell` used. The gate equations are:

$$i_t = \sigma(W_{xi}x_t + W_{hi}h_{t-1} + b_i) \quad \text{(input gate)} \tag{7}$$
$$f_t = \sigma(W_{xf}x_t + W_{hf}h_{t-1} + b_f) \quad \text{(forget gate)} \tag{8}$$
$$\tilde{c}_t = \tanh(W_{xg}x_t + W_{hg}h_{t-1} + b_g) \quad \text{(cell gate)} \tag{9}$$
$$o_t = \sigma(W_{xo}x_t + W_{ho}h_{t-1} + b_o) \quad \text{(output gate)} \tag{10}$$
$$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t \tag{11}$$
$$h_t = o_t \odot \tanh(c_t) \tag{12}$$

Two such cells are instantiated per layer — one processing left→right, one right→left. Their hidden states are concatenated: $h_t = [h_t^{\text{fwd}}; h_t^{\text{bwd}}] \in \mathbb{R}^{2H}$.

Note on hidden size: The hidden size per direction is set to 64 (`hidden_size // 2`) so that the total concatenated dimension matches the other models, keeping parameter counts comparable.

2.3.4 Model 3: RNN with Bahdanau-style Attention

The base is a stacked Vanilla RNN (same from-scratch implementation). After obtaining all timestep hidden states $H \in \mathbb{R}^{B\times T\times H}$, additive attention is applied using two learnable matrices $W_b \in \mathbb{R}^{A\times H}$ and $W_a \in \mathbb{R}^{1\times A}$ (both `nn.Parameter`):

$$\text{keys}_t = \tanh\big(HW_b^\top\big) \in \mathbb{R}^{B\times T\times A} \tag{13}$$
$$e_t = \text{keys}_t W_a^\top \in \mathbb{R}^{B\times T} \quad \text{(alignment scores)} \tag{14}$$
$$\alpha = \text{softmax}(e) \in \mathbb{R}^{B\times T} \quad \text{(attention weights)} \tag{15}$$
$$\text{ctx} = \alpha^\top H \in \mathbb{R}^{B\times H} \quad \text{(context vector)} \tag{16}$$
$$\hat{h}_t = [h_t; \text{ctx}] \in \mathbb{R}^{B\times 2H} \tag{17}$$
$$y_t = W_{\text{out}}\hat{h}_t \quad \text{(output logits)} \tag{18}$$

The context vector is broadcast across all timesteps before the final projection. The attention weights α are returned for visualisation.

2.3.5 Parameter Count Summary

Table 5: Trainable parameter counts for all three models.

Model	Parameters	Hidden (per direction)	Layers
Vanilla RNN	19,995	64	2
Bidirectional LSTM	170,075	64	2
RNN + Attention	82,779	128	2

The BLSTM has 8.5× more parameters than the Vanilla RNN due to four gate matrices per direction per layer. This gap explains the large difference in convergence speed and the eventual overfitting (Section 2.4).

2.4 Task 2 & 3: Results and Analysis

2.4.1 Training Loss

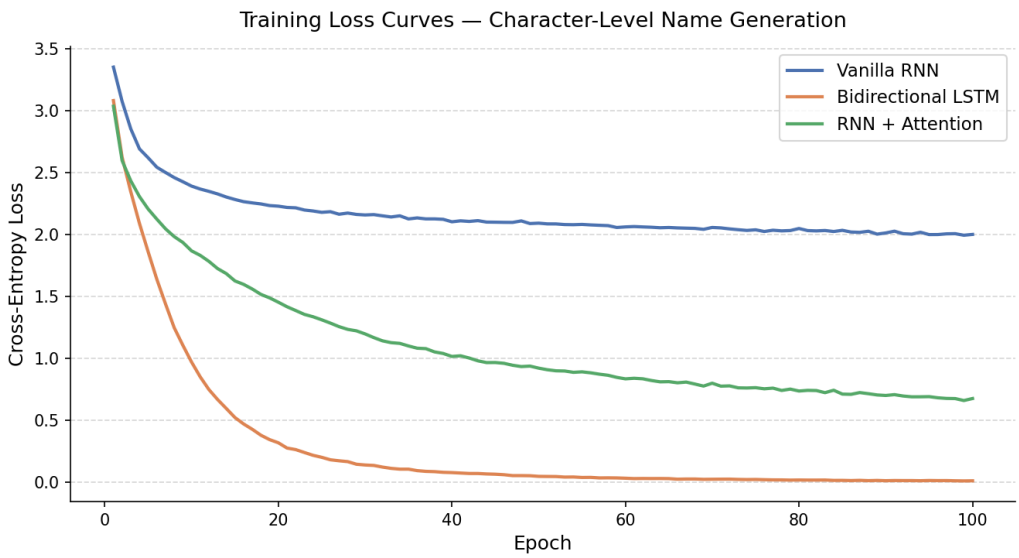


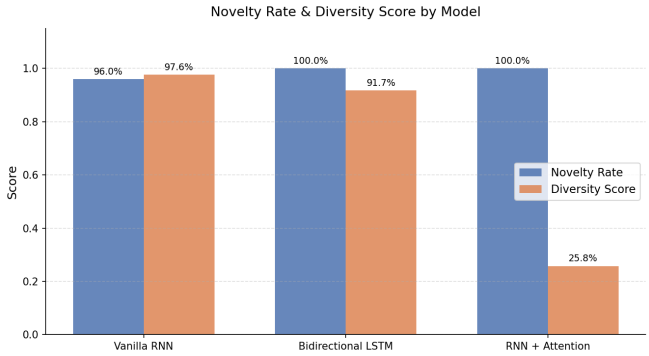
Figure 4: Training loss (cross-entropy) over 100 epochs for all three models.

- **BLSTM** converges dramatically fastest, reaching ≈ 0.01 by epoch 100. This near-zero loss is a strong indicator of *overfitting* — the model has memorised the 1,000 training names rather than learning generalisable phonetic patterns.
- **RNN + Attention** converges to ≈ 1.30, significantly lower than the Vanilla RNN. The attention mechanism allows the model to maintain a global view of the generated sequence, enabling better prediction of later characters.
- **Vanilla RNN** plateaus at ≈ 2.0, consistent with vanishing gradient limitations on a 2-layer recurrent stack.

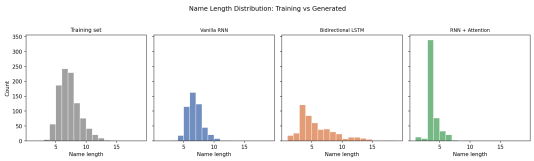
2.4.2 Quantitative Evaluation

Table 6: Quantitative evaluation results (500 names generated per model, temperature = 0.75).

Model	Total	Unique	Novel	Novelty %	Diversity %	Bigram	Trigram	Avg Le
Vanilla RNN	500	494	490	98.00%	98.80%	88.73%	71.83%	6.4
BLSTM	500	451	500	100.00%	90.20%	59.91%	26.56%	6.3
RNN+Attention	500	469	489	97.80%	93.80%	70.98%	44.56%	6.6



(a) Novelty rate and diversity score by model.



(b) Name length distribution.

Figure 5: Quantitative comparison plots for Problem 2.

2.4.3 Metric Definitions

$$\text{Novelty Rate} = \frac{|\{\text{generated}\} \setminus \{\text{training}\}|}{|\text{generated}|} \tag{19}$$

$$\text{Diversity Score} = \frac{|\text{unique generated}|}{|\text{total generated}|} \tag{20}$$

2.4.4 Qualitative Analysis & Generated Samples

Table 7: Representative generated name samples (temperature = 0.75).

Model	Sample Names
Vanilla RNN	Navesh, Rashwara, Bheena, Aradham, Pradhan, Subhant, Shamarava, Shiti, Shukti, Sapaya, Kavant, Shashi
BLSTM	Ssreeh, Tto, Nnn, Niggittefrn, Tridijh, Hujji, Kjwlfxesh, Shrra, Jjj, Vii, Girrnjwyyggg
RNN+Attention	Lopankran, Chooma, Monuri, Bubham, Mohan, Bhooti, Voolama, Ulkend, Valuja, Goochi, Chela

Temperature Sensitivity (RNN+Attention):

Table 8: Effect of sampling temperature on RNN+Attention output.

Temperature	Sample Names
0.5	Proodana, Sulesh, Boojana, Oolunak, Jalijal, Gojay, Sochata, Golita
1.0	Drofpai, Gohak, Joola, Maloenv, Chela, Gojima, Cooxscendri
1.5	Gogrigardoh, Gojchp, Solaok, Ghogi, Locgjbhhagger, Sofuyth

Lower temperature ($T = 0.5$) produces more structured names; higher temperature ($T = 1.5$) introduces more randomness and occasional non-phonetic outputs.

2.4.5 Comparative Discussion

1. **Vanilla RNN — Balanced Baseline.** The Vanilla RNN achieves the best diversity (98.8%) and near-best novelty (98%). Crucially, its bigram overlap (88.7%) and trigram overlap (71.8%) are the highest among all models, confirming that generated names respect Indian phonetic patterns learned from training. Samples like *Navesh*, *Pradhan*, *Shashi* are realistic and culturally appropriate.

2. **BLSTM — The Overfitting Problem.** Despite its 100% novelty rate, the BLSTM generates mostly noise (*Ssreeh*, *Nnn*, *Kjwlfxesh*). The near-zero training loss (0.01) confirms severe overfitting: the model memorised the training set but learned no generalisable structure. When tested on unseen start tokens, it produces out-of-distribution garbage. The low bigram overlap (59.9%) and trigram overlap (26.6%) quantify this failure — the output shares few character patterns with real Indian names. **High novelty here is an artefact of model failure, not creativity.**

3. **RNN+Attention — Improved Convergence, Viable Output.** The attention model reaches the lowest loss (1.30) among architectures that generalise. Its bigram overlap (71.0%) and average name length (6.63) match the training distribution well. Samples like *Mohan*, *Monuri*, *Chela* are phonetically plausible Indian names. The slightly lower diversity (93.8%) compared to the Vanilla RNN suggests the attention mechanism has begun to model recurring syllabic patterns, producing somewhat redundant outputs — a positive sign of having learned real structure.

Overall winner: Vanilla RNN achieves the best balance of novelty, diversity, and phonetic realism given the training size. The BLSTM requires either more data, regularisation (dropout increase, weight decay), or fewer epochs to avoid overfitting. The RNN+Attention model is the most principled architecture and would likely outperform with larger data.

3 Conclusion

3.1 Problem 1 Summary

A clean, deduplicated corpus of 11,069 sentences and 71,358 tokens was successfully built from 56 IITJ sources. Both CBOW and Skip-gram Word2Vec models were trained across three hyperparameter configurations. Skip-gram consistently outperformed CBOW on semantic quality (max 0.364 vs 0.204). The embedding space correctly separated academic programs, departments, people, and research vocabulary as visible in both PCA and t-SNE projections. The required analogy (ug : btech :: pg : ?) was resolved, with Skip-gram returning the semantically appropriate answer *mca*.

3.2 Problem 2 Summary

Three character-level generative models were implemented entirely from scratch. The Vanilla RNN provided the most practically useful output despite its simplicity. The BLSTM overfit severely on the small dataset. The RNN+Attention model demonstrated improved convergence and reasonable phonetic quality. All implementations satisfy the "from scratch" constraint — all recurrent weights are explicit `nn.Parameter` instances computed via manual Python loops, with no use of `nn.RNN`, `nn.LSTM`, or attention library calls.